

# ***Operating System (ISE3137)***

**Jump Together, Fly Farther!**



인하대학교 국제학부

## ***Week 12 Lecture***

**ISE Department  
Prof. Mehdi Pirahandeh**

# Chapter 10: Virtual Memory



# Objectives

---

- Define virtual memory and describe its benefits.
- Illustrate how pages are loaded into memory using demand paging.
- Apply the FIFO, optimal, and LRU page-replacement algorithms.
- Describe the working set of a process, and explain how it is related to program locality.
- Describe how Linux manages virtual memory.
- Design a virtual memory manager simulation in the C programming language.

# Background

---

- Code needs to be in memory to execute, but the entire program is rarely used
- Entire program code not needed at the same time
- Consider the ability to execute the partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running
  - Less I/O needed to load or swap programs into memory

# Virtual memory

---

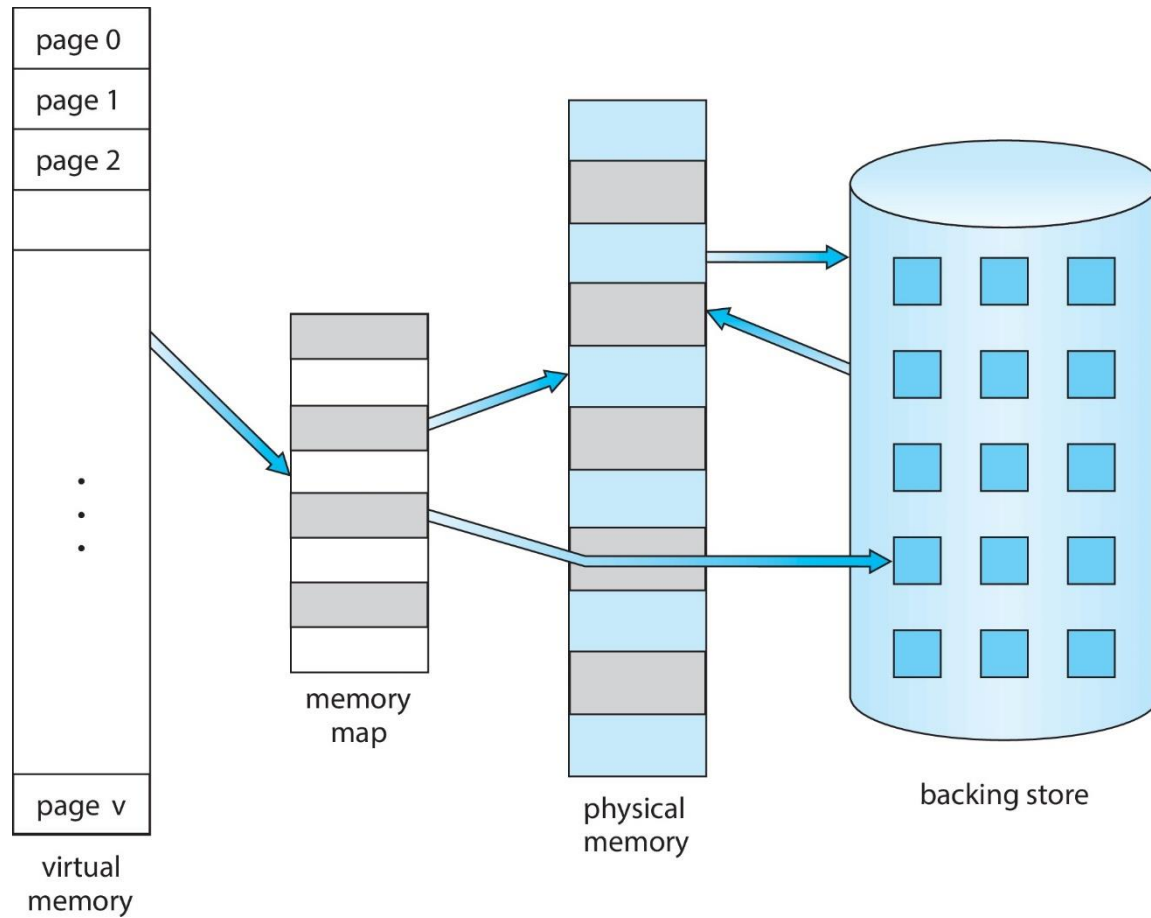
- **Virtual memory** – separation of user logical memory from physical memory
  - Only part of the program needs to be in memory for execution
  - Logical address space can therefore be much larger than physical address space
  - Allows address spaces to be shared by several processes

# Virtual memory (Cont.)

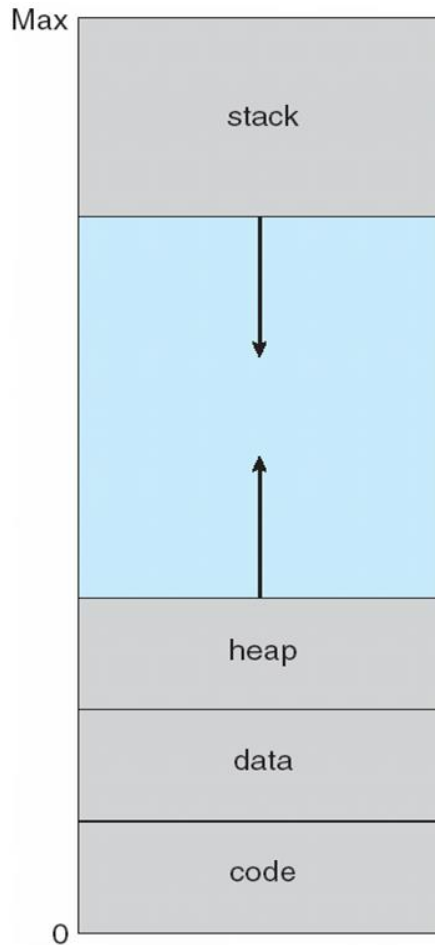
---

- **Virtual address space** – logical view of how the process is stored in memory
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

# Virtual Memory That is Larger Than Physical Memory



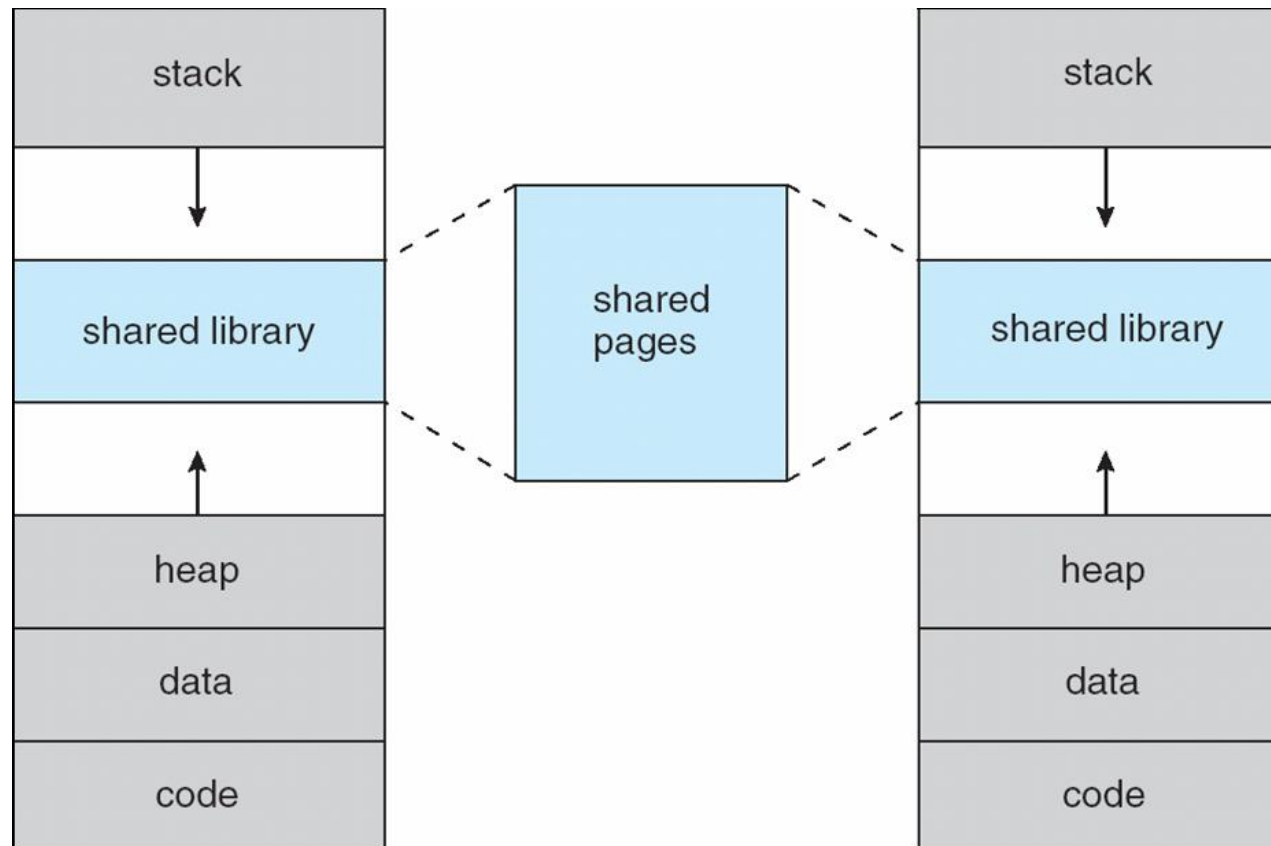
# Virtual-address Space



- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during `fork()`, speeding process creation



# Shared Library Using Virtual Memory y



# Demand Paging



Could bring entire process into memory at load time



Or bring a page into memory only when it is needed



Similar to paging system with swapping (diagram on right)



Page is needed  $\Rightarrow$  reference to it

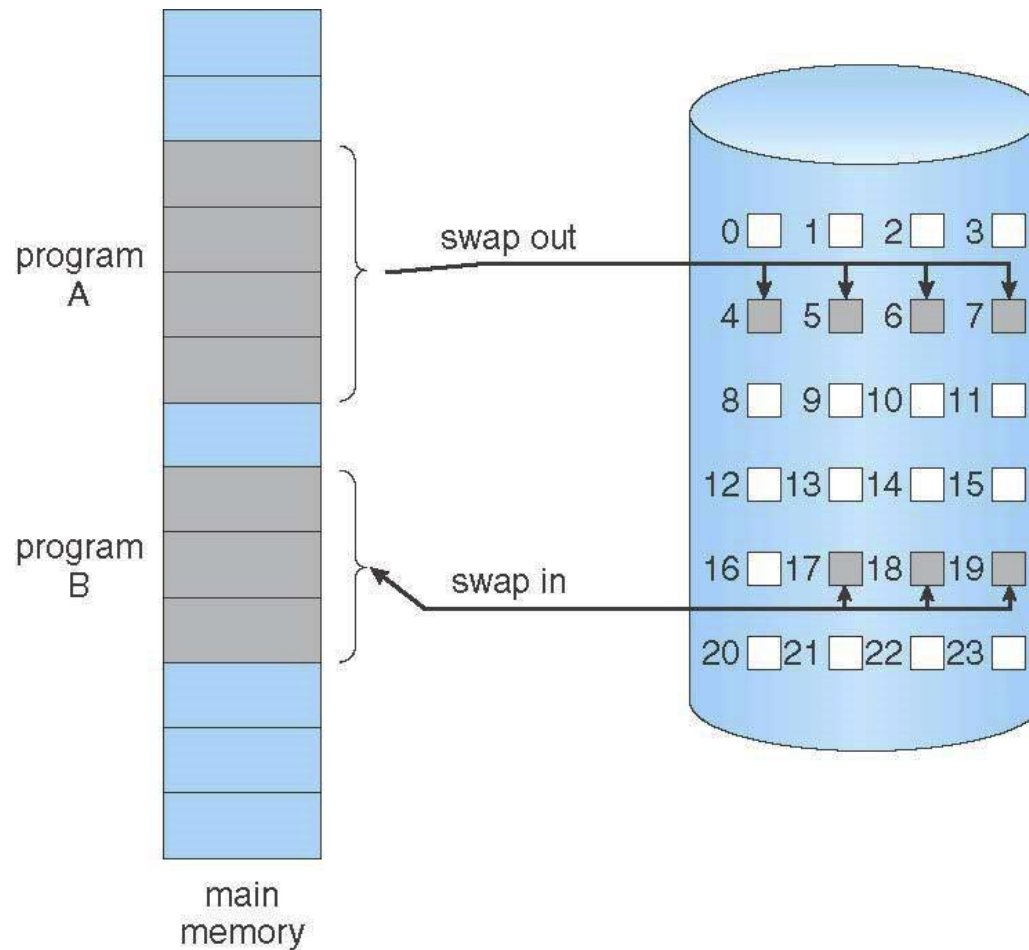
invalid reference  $\Rightarrow$  abort

not-in-memory  $\Rightarrow$  bring to memory

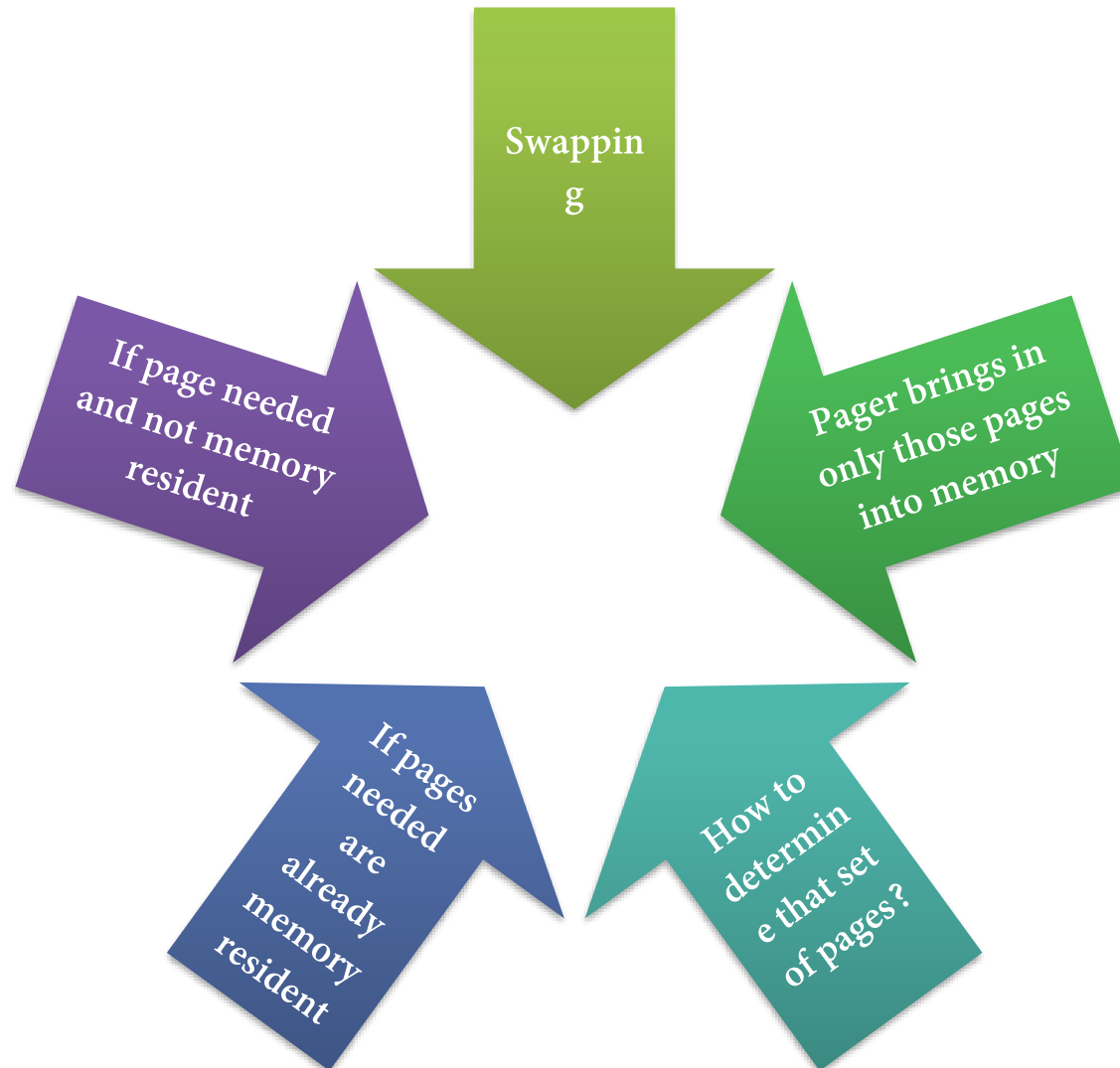


**Lazy swapper** – never swaps a page into memory unless page will be needed

# Demand Paging



# Basic Concepts



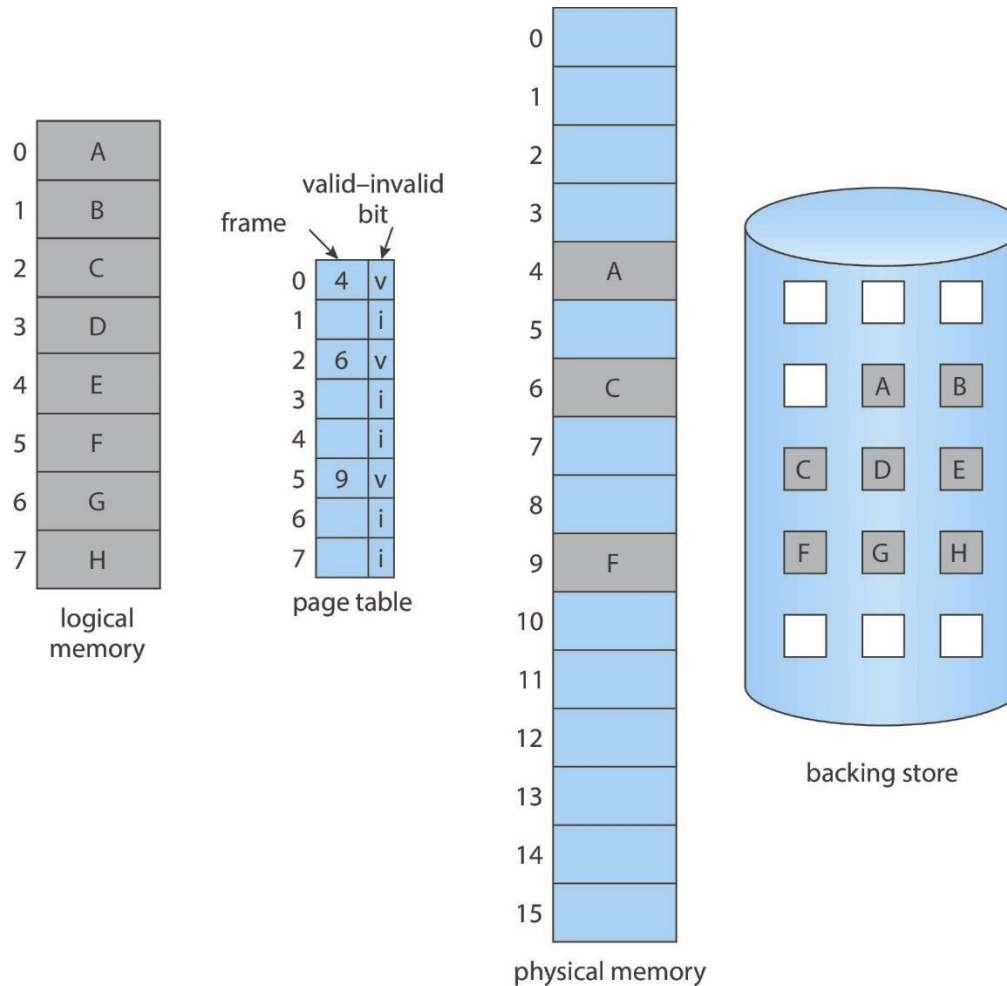
# Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated  
(**v**  $\Rightarrow$  in-memory – **memory resident**, **i**  $\Rightarrow$  not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snaps hot:

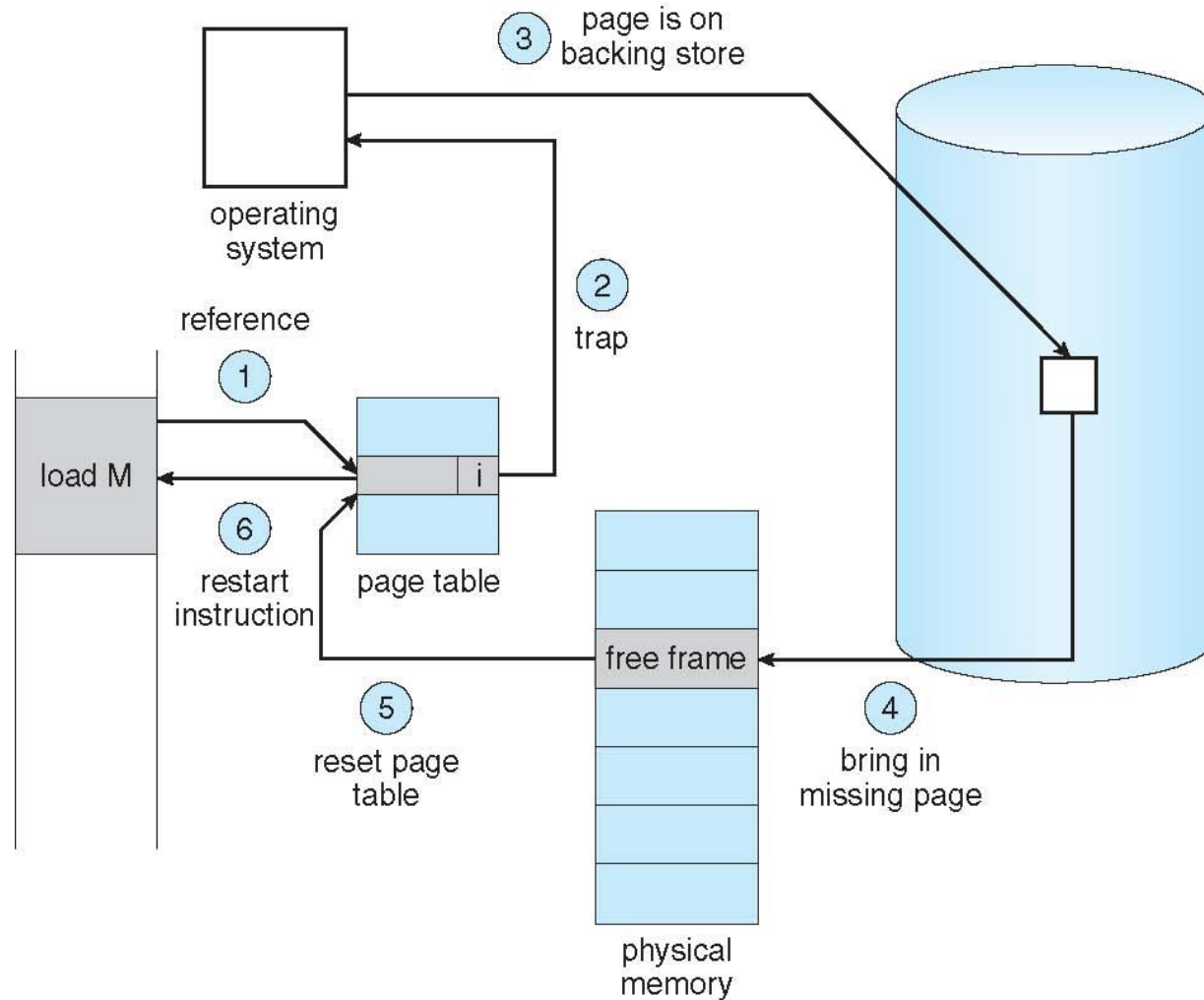
| Frame # | valid-invalid bit |
|---------|-------------------|
|         |                   |
|         | <b>v</b>          |
|         | <b>v</b>          |
|         | <b>v</b>          |
|         | <b>i</b>          |
| ...     |                   |
|         | <b>i</b>          |
|         | <b>i</b>          |

page table

# Page Table When Some Pages Are Not in Main Memory



# Steps in Handling a Page Fault (Cont.)



# Aspects of Demand Paging

---



Extreme case –  
start process with  
*no* pages in  
memory



Actually, a given  
instruction could  
access multiple  
pages -> multiple  
page faults

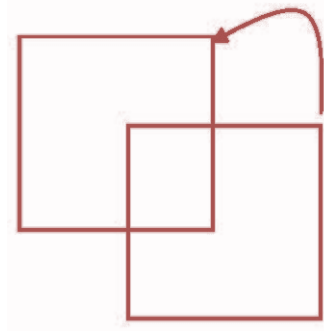


Hardware support  
needed for demand  
paging



# Instruction Restart

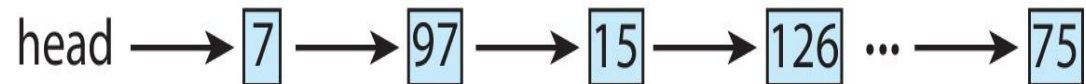
- Consider an instruction that could access several different locations
  - Block move



- Auto increment/decrement location
  - Restart the whole operation?

# Free-Frame List

- A page fault occurs
- Most operating systems maintain a **free-frame list**



- Operating system typically allocates free frames using a technique known as **zero-fill-on-demand**
- When a system starts up

# Stages in Demand Paging – Worse Case

Trap to the  
operating system

Save the user  
registers and  
process the state

Determine that the  
interrupt was a  
page fault

Check that the  
page reference is  
legal

Issue a read from  
the disk to a free  
frame

While waiting,  
allocate the CPU to  
some other user

Receive an  
interrupt from the  
disk I/O subsystem  
(I/O completed)

Save the registers  
and process state  
for the other user

Determine that the  
interrupt was from  
the disk

Correct the page  
table and other  
tables to show page  
is now in memory

Wait for the CPU  
to be allocated to  
this process again

Restore the user  
registers, process  
state, and etc

# Performance of Demand Paging



인하대학교  
INHA UNIVERSITY

- ✓ Three major activities
  - Service the interrupt
  - Read the page
  - Restart the process
- Page Fault Rate  $0 \leq p \leq 1$
- Effective Access Time (EAT)

$$\text{EAT} = (1 - p) \times \text{memory access} \\ + p (\text{page fault overhead} + \text{swap page out} + \text{swap page in})$$

# Demand Paging Example

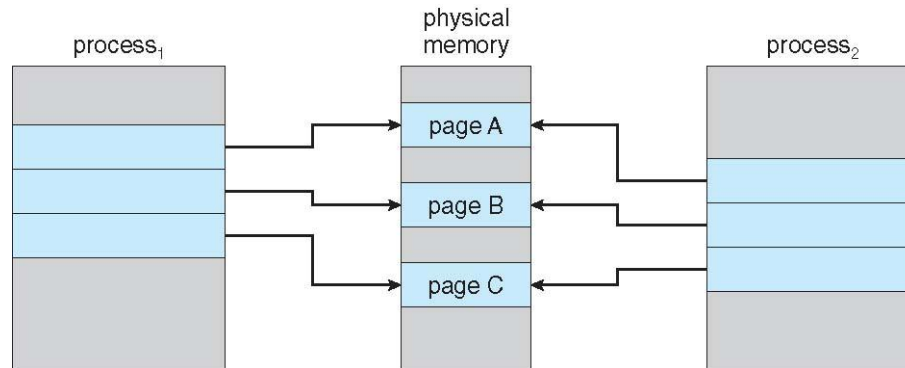
- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- $EAT = (1 - p) \times 200 + p (8 \text{ milliseconds})$   
 $= (1 - p) \times 200 + p \times 8,000,000$   
 $= 200 + p \times 7,999,800$
- If one access out of 1,000 causes a page fault, then  
EAT = 8.2 microseconds.  
This is a slowdown by a factor of 40!!
- If want performance degradation < 10 percent
  - $220 > 200 + 7,999,800 \times p$   
 $20 > 7,999,800 \times p$
  - $p < .0000025$
  - < one page fault in every 400,000 memory accesses

# Copy-on-Write

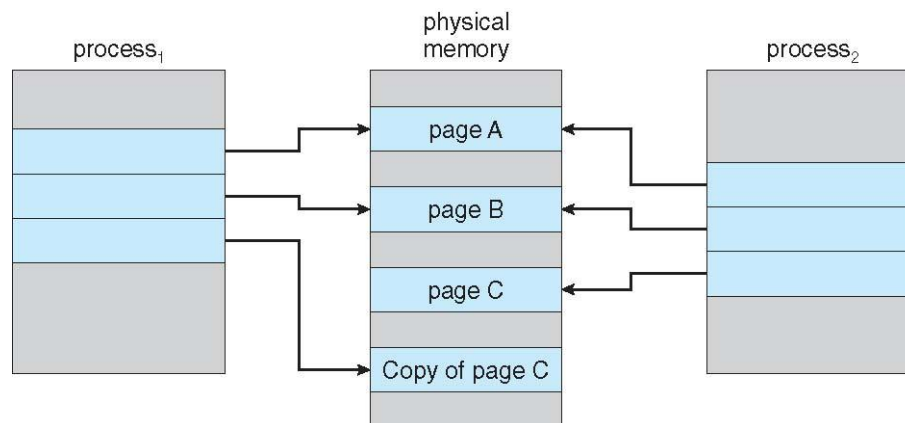
- **Copy-on-Write** (COW) allows both parent and child processes to initially share the same pages in memory.
  - If either process modifies a shared page, only then is the page copied
- ✓ COW allows more efficient process creation as only modified pages are copied
- ✓ In general, free pages are allocated from a **pool** of **zero-fill-on-demand** pages

# Modifying Page C

## Before Process 1 Modifies Page C



## After Process 1 Modifies Page C



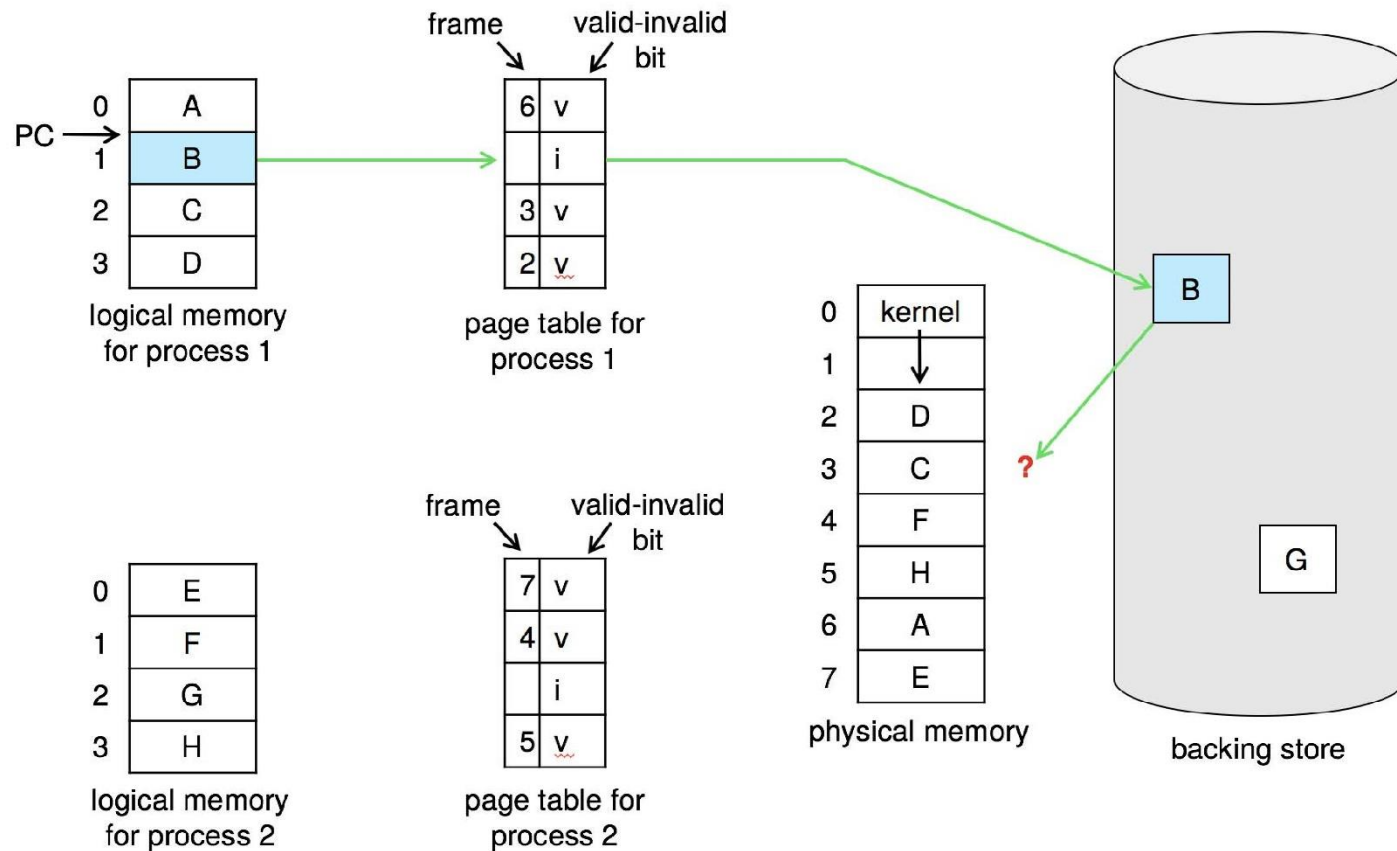
# Page Replacement

---

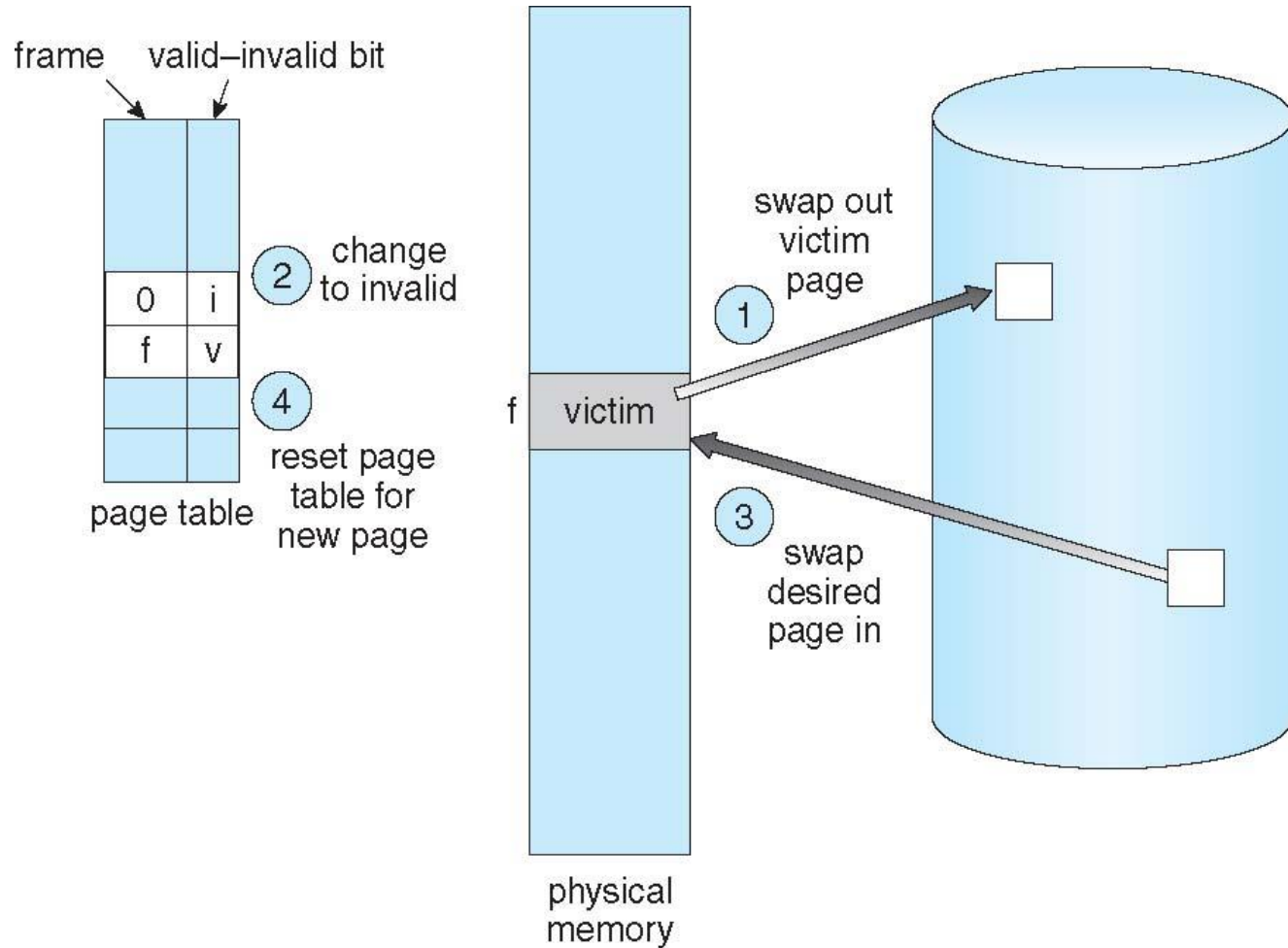
- Prevent **over-allocation** of memory
- Using the **modify (dirty) bit**



# Need For Page Replacement



# Page Replacement



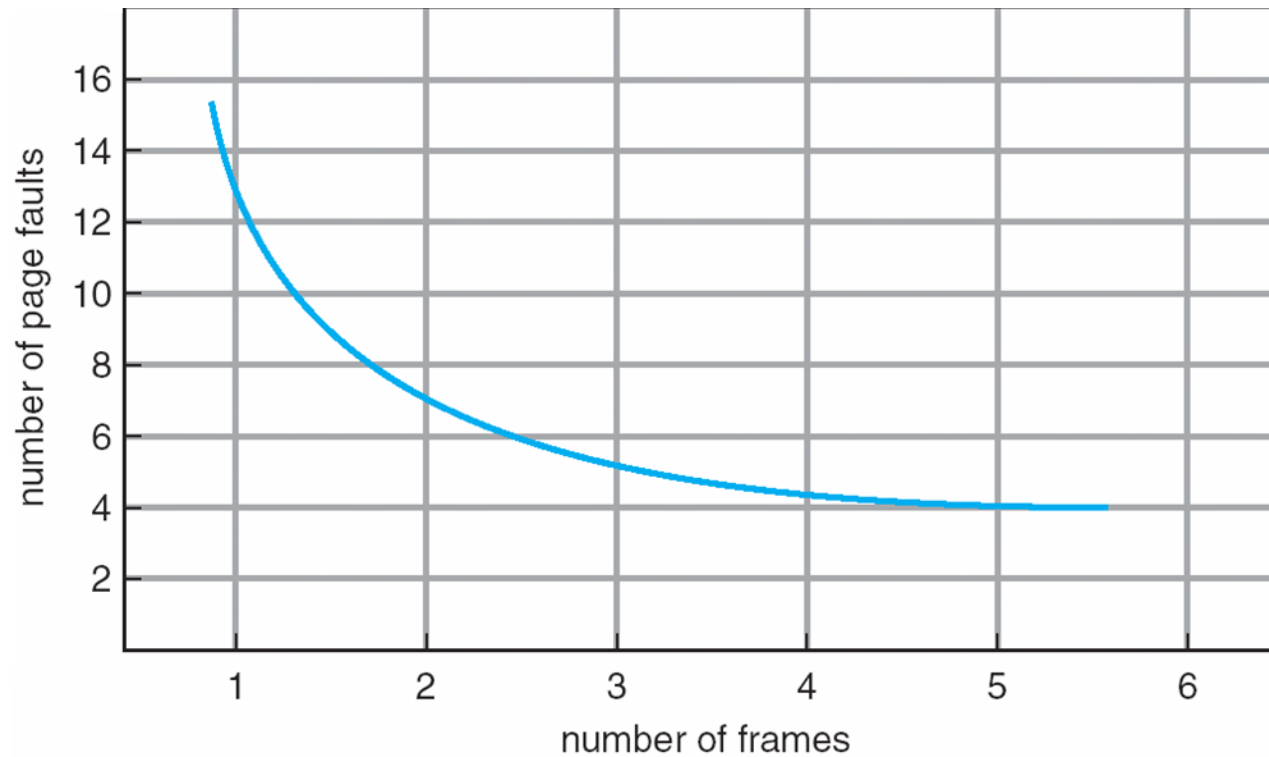
# Page and Frame Replacement Algorithms



인하대학교  
INHA UNIVERSITY

- **Frame-allocation algorithm** determines
  - **Page-replacement algorithm**
- Evaluate the algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string

# Graph of Page Faults Versus the Number of Frames



# First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

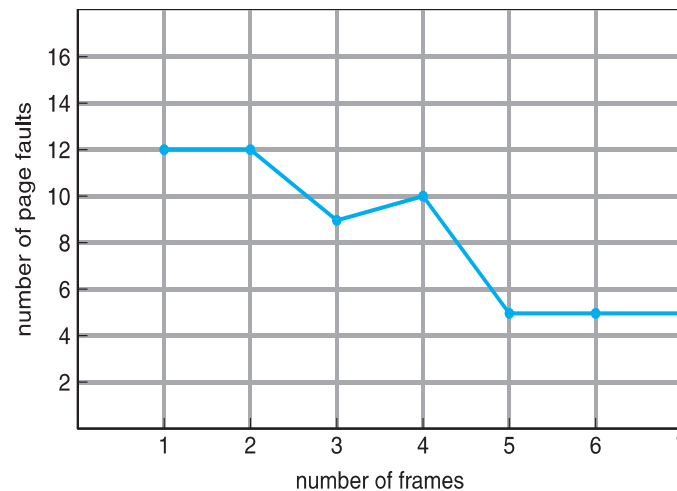
reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7 | 7 | 7 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 | 0 | 0 | 7 | 7 | 7 |
|   | 0 | 0 | 0 | 3 | 3 | 3 | 2 | 2 | 2 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
|   |   | 1 | 1 | 1 | 0 | 0 | 0 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 | 1 |

15 page faults

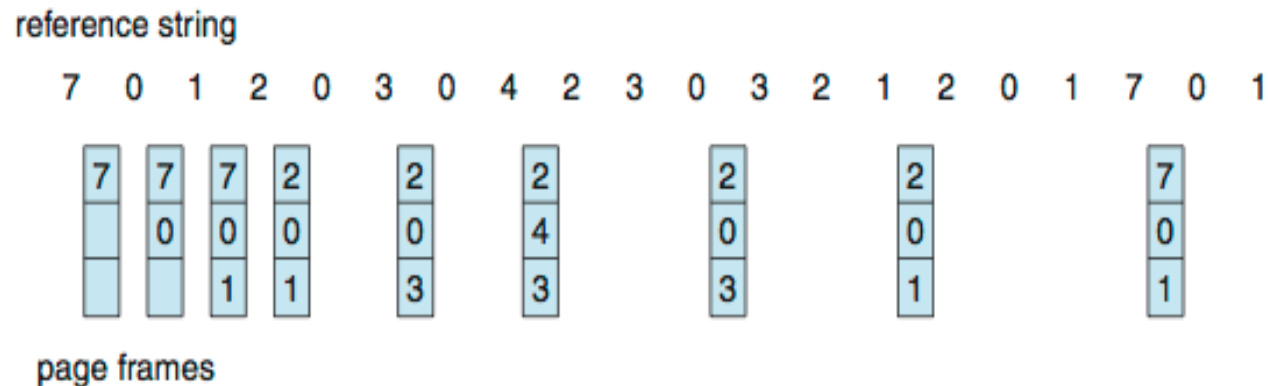
page frames



Belady's Anomaly

# Optimal Algorithm

- Replace page that will not be used for longest period of time
  - 9 is optimal for the example
- How do you know this?
  - Can't read the future
- Used for measuring how well your algorithm performs



# Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

|   |   |   |   |  |  |  |   |  |   |   |   |   |  |  |  |   |  |   |   |
|---|---|---|---|--|--|--|---|--|---|---|---|---|--|--|--|---|--|---|---|
| 7 | 7 | 7 | 2 |  |  |  | 2 |  | 4 | 4 | 4 | 0 |  |  |  | 1 |  | 1 | 1 |
|   | 0 | 0 | 0 |  |  |  | 0 |  | 0 | 0 | 3 | 3 |  |  |  | 3 |  | 0 | 0 |
|   |   | 1 | 1 |  |  |  | 3 |  | 3 | 2 | 2 | 2 |  |  |  | 2 |  | 2 | 7 |

page frames

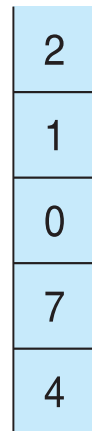
- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

# LRU Algorithm (Cont.)

- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly
- Use Of A Stack to Record Most Recent Page References

reference string

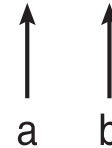
4 7 0 7 1 0 1 2 1 2 7 1 2



stack  
before  
a



stack  
after  
b





# LRU Approximation Algorithms

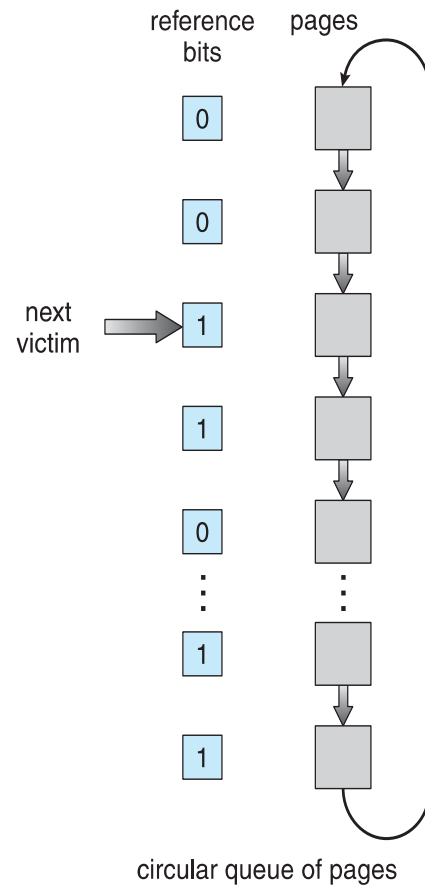
---



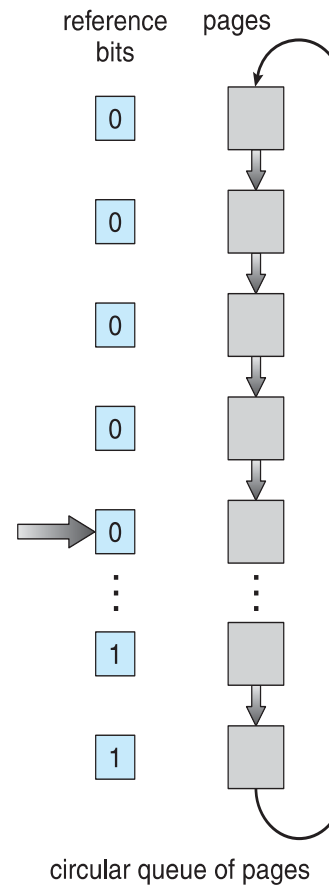
인하대학교  
INHA UNIVERSITY

- LRU needs special hardware and still slow
- **Reference bit**
  - With each page associate a bit, initially = 0
  - When page is referenced bit set to 1
  - Replace any with reference bit = 0 (if one exists)
    - We do not know the order, however

# Second-chance Algorithm



(a)



(b)

# Counting Algorithms

---

- Keep a counter of the number of references that have been made to each page
  - Not common
- **Least Frequently Used (LFU) Algorithm:**
  - Replaces page with smallest count
- **Most Frequently Used (MFU) Algorithm:**
  - Based on the argument that the page with the smallest count was probably just brought in and has yet to be used

# Page-Buffering Algorithms

---

- Keep a pool of free frames, always
  - Then frame available when needed, not found at fault time
  - Read page into free frame and select victim to evict and add to free pool
  - When convenient, evict victim
- Possibly, keep list of modified pages
  - When backing store otherwise idle, write pages there and set to non-dirty
- Possibly, keep free frame contents intact and note what is in them
  - If referenced again before reused, no need to load contents again from disk
  - Generally useful to reduce penalty if wrong victim frame selected

# Applications and Page Replacement

- All of these algorithms have OS guessing about future page access
- Some applications have better knowledge – i.e. databases
- Memory intensive applications can cause double buffering
  - OS keeps copy of page in memory as I/O buffer
  - Application keeps page in memory for its own work
- Operating system can given direct access to the disk, getting out of the way of the applications
  - **Raw disk** mode
- Bypasses buffering, locking, etc.

# Allocation of Frames

- Each process needs **minimum** number of frames
- Example: IBM 370 – 6 pages to handle SS MOVE instruction:
  - instruction is 6 bytes, might span 2 pages
  - 2 pages to handle *from*
  - 2 pages to handle *to*
- **Maximum** of course is total frames in the system
- Two major allocation schemes
  - fixed allocation
  - priority allocation
- Many variations

# Fixed Allocation

- Equal allocation – For example, if there are 100 frames (after allocating frames for the OS) and 5 processes, give each process 20 frames
  - Keep some as free frame buffer pool
- Proportional allocation – Allocate according to the size of process
  - Dynamic as degree of multiprogramming, process sizes change

–  $s_i$  = size of process  $p_i$

–  $S = \sum s_i$

–  $m$  = total number of frames

–  $a_i$  = allocation for  $p_i = \frac{s_i}{S} \times m$

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \cdot 62 \gg 4$$

$$a_2 = \frac{127}{137} \cdot 62 \gg 57$$

# Global vs. Local Allocation

---

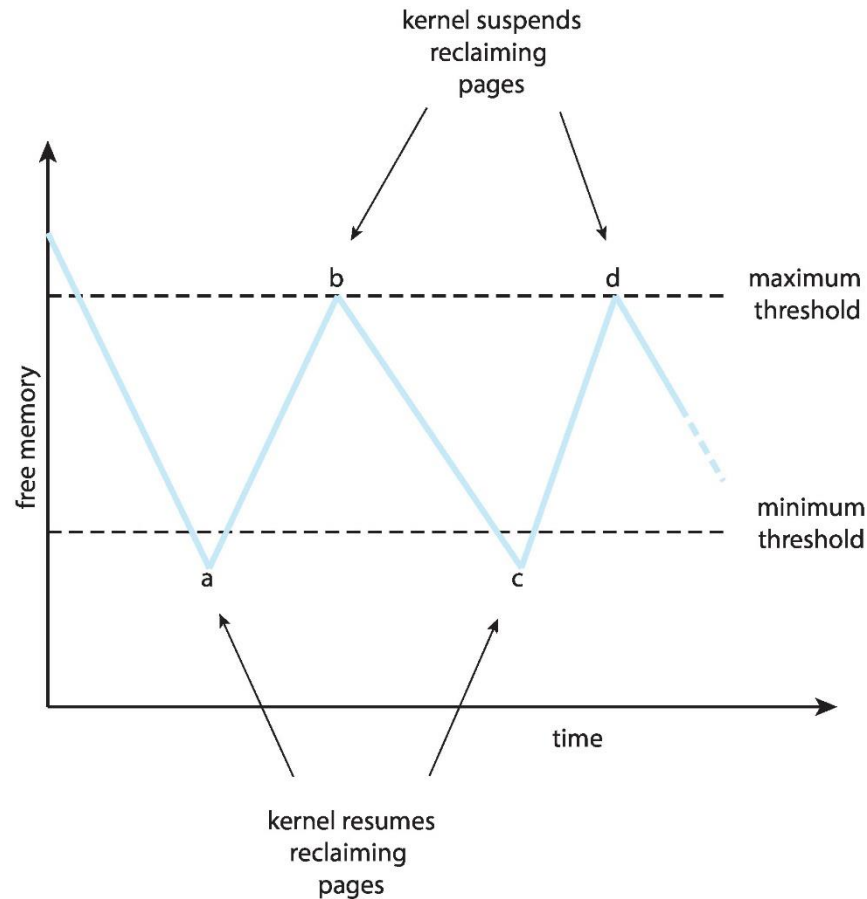
- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
  - But then process execution time can vary greatly
  - But greater throughput so more common
- **Local replacement** – each process selects from only its own set of allocated frames
  - More consistent per-process performance
  - But possibly underutilized memory



# Reclaiming Pages

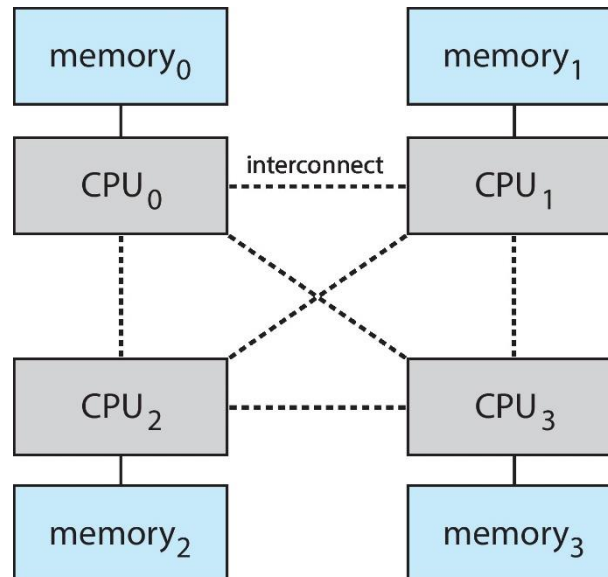
- A strategy to implement global page-replacement policy
- All memory requests are satisfied from the free-frame list, rather than waiting for the list to drop to zero before we begin selecting pages for replacement,
- Page replacement is triggered when the list falls below a certain threshold.
- This strategy attempts to ensure there is always sufficient free memory to satisfy new requests.

# Reclaiming Pages Example



# Non-Uniform Memory Access

- So far, we assumed that all memory accessed equally
- Many systems are **NUMA** – speed of access to memory varies
  - Consider system boards containing CPUs and memory, interconnected over a system bus
- NUMA multiprocessing architecture



# Non-Uniform Memory Access (Cont.)

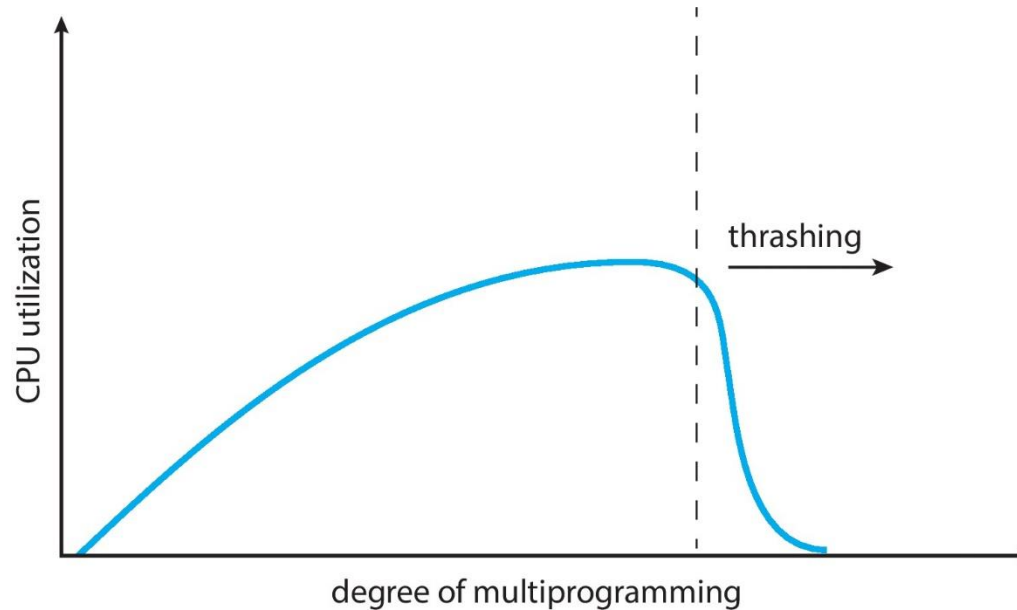
- Optimal performance comes from allocating memory “close to” the CPU on which the thread is scheduled
  - And modifying the scheduler to schedule the thread on the same system board when possible
  - Solved by Solaris by creating **lggroups**
    - Structure to track CPU / Memory low latency groups
    - Used my schedule and pager
    - When possible schedule all threads of a process and allocate all memory for that process within the lgroup

# Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high
  - Page fault to get page
  - Replace existing frame
  - But quickly need replaced frame back
  - This leads to:
    - Low CPU utilization
    - Operating system thinking that it needs to increase the degree of multiprogramming
    - Another process added to the system

# Thrashing (Cont.)

- **Thrashing.** A process is busy swapping pages in and out



# Demand Paging and Thrashing



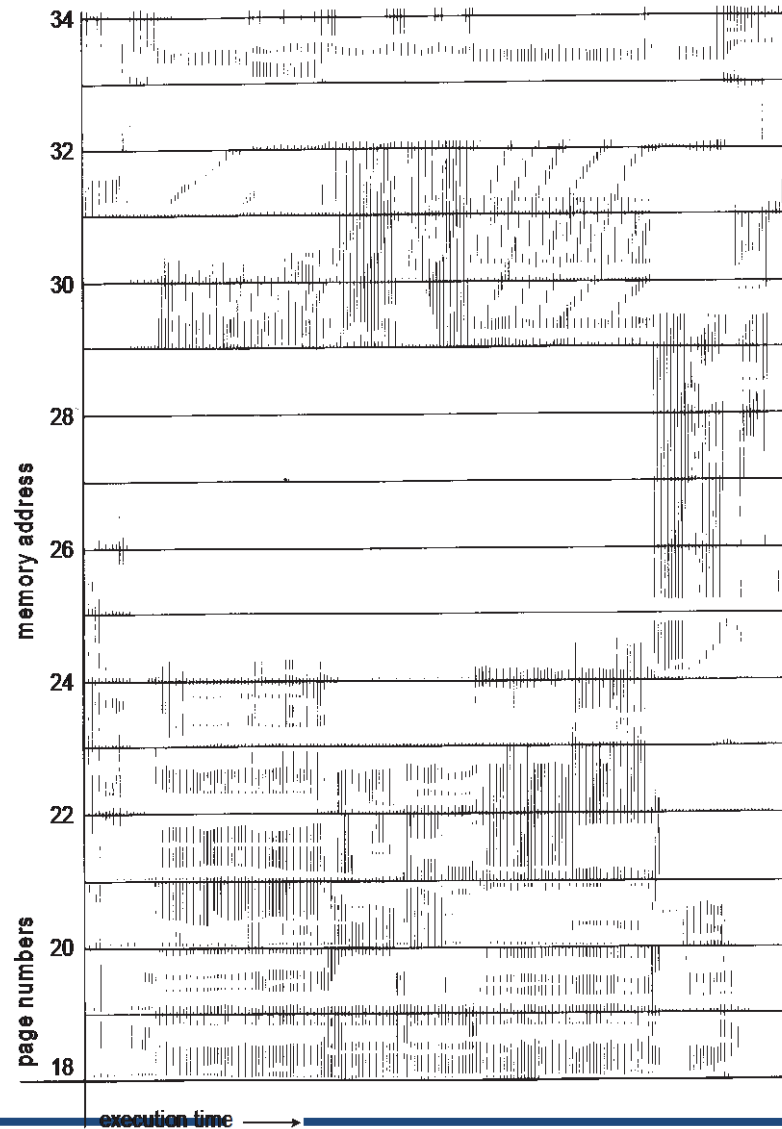
인하대학교  
INHA UNIVERSITY

- Why does demand paging work?

## Locality model

- Process migrates from one locality to another
  - Localities may overlap
- Why does thrashing occur?
  - $\Sigma$  size of locality > total memory size
- Limit effects by using local or priority page replacement

# Locality In A Memory-Reference Pattern





# Working-Set Model

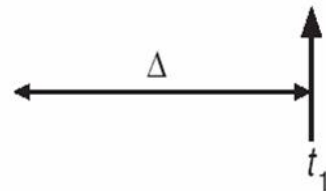
- $\Delta \equiv$  working-set window  $\equiv$  a fixed number of page references  
Example: 10,000 instructions
- $WSS_i$  (working set of Process  $P_i$ ) = total number of pages referenced in the most recent  $\Delta$  (varies in time)
  - if  $\Delta$  too small will not encompass entire locality
  - if  $\Delta$  too large will encompass several localities
  - if  $\Delta = \infty \Rightarrow$  will encompass entire program
- $D = \sum WSS_i \equiv$  total demand frames
  - Approximation of locality

# Working-Set Model (Cont.)

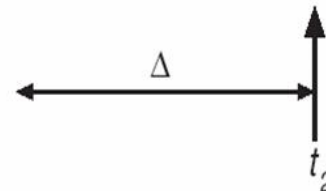
- if  $D > m \Rightarrow$  Thrashing
- Policy if  $D > m$ , then suspend or swap out one of the processes

page reference table

... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



$$WS(t_1) = \{1, 2, 5, 6, 7\}$$



$$WS(t_2) = \{3, 4\}$$

# Keeping Track of the Working Set

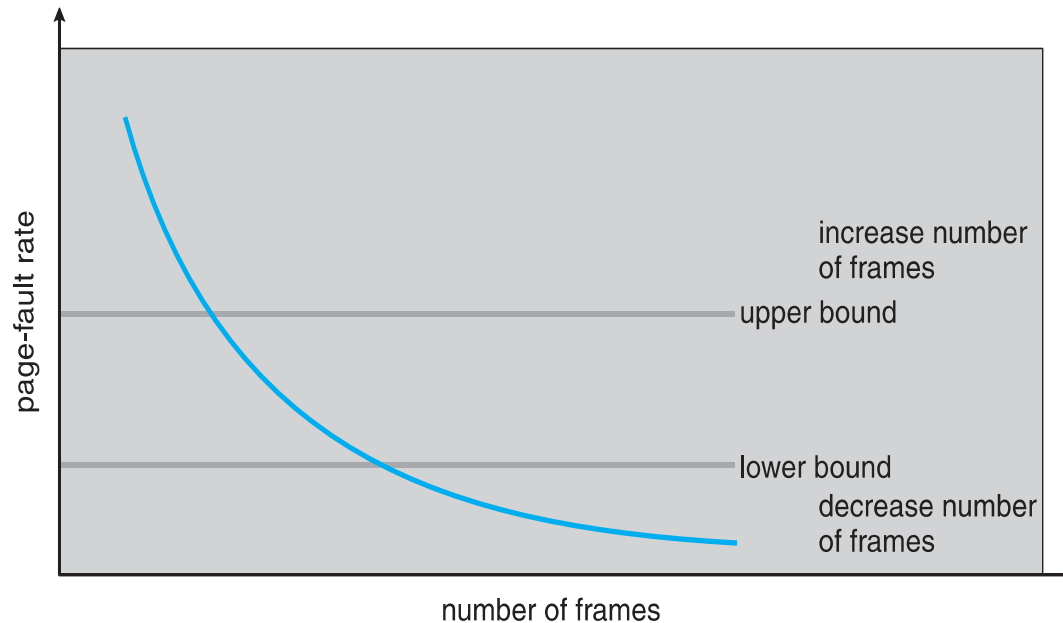


인하대학교  
INHA UNIVERSITY

- Approximate with interval timer + a reference bit
- Example:  $\Delta = 10,000$ 
  - Timer interrupts after every 5000 time units
  - Keep in memory 2 bits for each page
  - Whenever a timer interrupts copy and sets the values of all reference bits to 0
  - If one of the bits in memory = 1  $\Rightarrow$  page in working set
- Why is this not completely accurate?
- Improvement = 10 bits and interrupt every 1000 time units

# Page-Fault Frequency

- More direct approach than WSS
- Establish “acceptable” **page-fault frequency (PFF)** rate and use local replacement policy
  - If actual rate too low, process loses frame
  - If actual rate too high, process gains frame

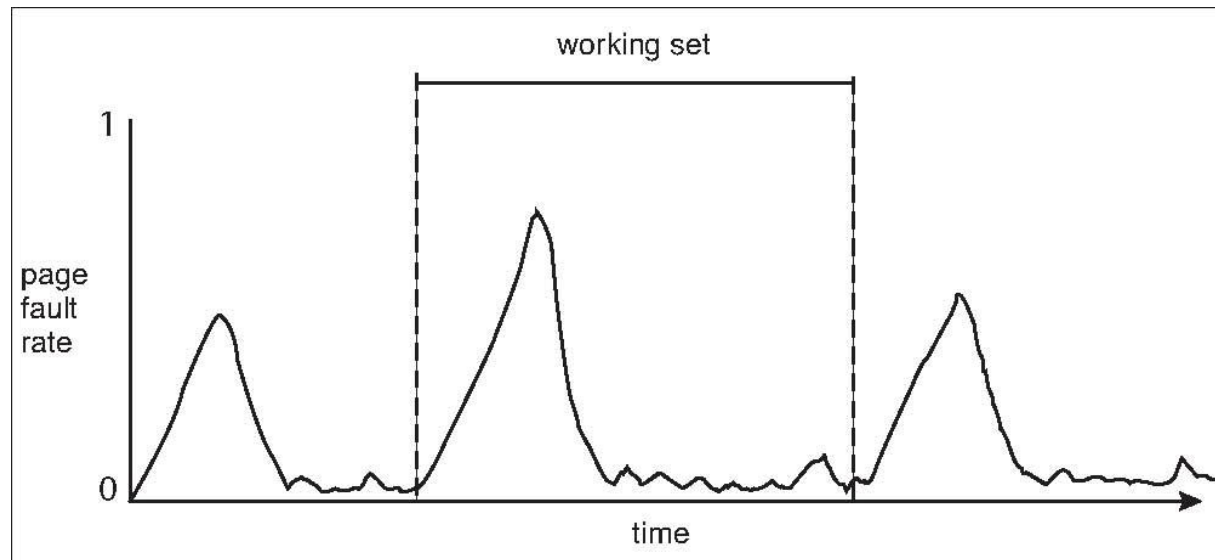


# Working Sets and Page Fault Rates



인하대학교  
INHA UNIVERSITY

- Direct relationship between working set of a process and its page-fault rate
- Working set changes over time
- Peaks and valleys over time



# Allocating Kernel Memory

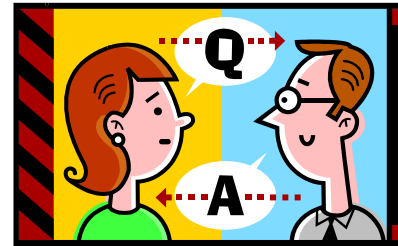
---

- Treated differently from user memory
- Often allocated from a free-memory pool
  - Kernel requests memory for structures of varying sizes
  - Some kernel memory needs to be contiguous
    - i.e., for device I/O

# Brief break (if on schedule)

---

## Q&A?



Prof. Mehdi Pirahandeh  
E-mail : mehdi@inha.ac.kr